

TTT 와 Titans 의 효율

Scaling 병목과 해법 , 그리고 memory device 집중 조명 · 근거등급 a= 명시 · b= 추론 · c= 미명시

목적 · 범위 · 읽는 법

목적

TTT / Titans 계열을 '효율 · scaling' 시스템 관점으로 집중 조명
세 scaling 병목 (model-size · training batch · serving P/D) 과
각 해법 (무엇을 · 어떻게 · 왜 working), 그리고 memory device 한계 · 개선

3 가지 caveat

- ① 아키텍처 세부가 아니라 효율 · scaling 관점
- ② 2026 최신 多 → 수치 · 재현성 미확정 → 근거등급
- ③ 단일 논문은 불완전 — 갈래는 갈래대로

흐름

I TTT·Titans 집중 조명
II 병목 3 종 → III 해법 3 종
IV memory device 한계 · 개선
→ 종합 지도 · 열린 질문

Part I

TTT · Titans 집중 조명

은닉상태가 곧 ' 학습 중인 모델 '

- 은닉상태를 작은 모델의 weights W_t 로 보고 , 매 토큰 self-supervised gradient 1 스텝으로 갱신 [a]
 - 출력은 그 갱신된 모델의 forward: $z_t = f_{\{W_t\}}(q_t)$. f 가 선형 =TTT-Linear, 2-layer MLP=TTT-MLP [a]
- 메모리 용량이 ' 벡터 크기 ' 가 아니라 ' 모델 가중치 수 ' 로 스케일 [a]
 - → 문맥이 길어질수록 손실 계속 감소 (Mamba 는 ~16k 에서 정체) [a]
- 뿌리 : fast-weight programmer(Schlag) — linear attention $\equiv$ fast weight, delta rule = gradient step [a]

$$W_t = W_{t-1} - \eta \nabla_W \ell(W_{t-1}; x_t), \quad \ell = \|f_W(k_t) - v_t\|^2$$

순차성 = 모든 병목의 뿌리

이중성 (duality) 이 효율의 열쇠

같은 연산의 세 형태 :

parallel(학습) · recurrent($O(1)$ 추론) · chunkwise

TTT 의 dual form: 청크 내 gradient 들을 하나의

큰 행렬곱으로 묶음 (primal 대비 $>5\times$, JAX 기준) [a]

청크 내부는 병렬 , W 의 이월은 청크 경계에서만 [b]

그러나 순차성은 남는다

W_t 는 W_{t-1} 에 의존 $\rightarrow$ 본질적으로 순차 [b]

chunkwise 는 순차성을 ' 청크 경계로 밀어낼 ' 뿐

작은 청크 (16-64 토큰) $\rightarrow$ FLOPs util $< 5\%$ (LaCT) [a]

큰 청크 $\rightarrow$ util $\uparrow$ 이나 세밀한 적응력 $\downarrow$ (trade-off) [b]

$\rightarrow$ 이 trade-off 가 이후 모든 병목을 낳는다

deep neural memory + surprise · momentum · forget

- long-term memory M = deep MLP; 각 토큰에서 M 의 가중치를 test time 에 갱신 [a]
 - 갱신 신호 = associative loss $\| M(k_t) - v_t \|^2$ 의 gradient = 'surprise' [a]
 - 여기에 momentum(과거 surprise 누적) + weight-decay(adaptive forget) 결합 [a]
- 3 가지 통합 방식 : MAC(memory as context) · MAG(as gate) · MAL(as layer) [a]
- attention(단기 정확) + neural memory(장기 압축) 의 하이브리드 [a]
 - chunk-parallel 학습으로 순차 갱신을 가속 [a]

효율 라인 Atlas·TNT·HOPE·Sleep — 그리고 비판

효율 · 표현력 확장 라인

Atlas: Omega rule(문맥 단위)+Muon(2 차)+ 다항 feature 용량 [a]

TNT: chunkwise training 가속 — global chunk + local 병렬 [a]

HOPE / Nested Learning: optimizer=memory · CMS ·
self-modifying Titans [a]

Sleep: offline consolidation — parameter 성장 +
Knowledge Seeding + Dreaming(RL) [a]

⚠ skeptic — Titans Revisited

경량 재구현으로 재현성 · 실기여를 비판적 검증 [a]

neural memory 는 유효하나 ,
chunking 때문에 baseline 을 ' 항상 ' 넘지는 못함 [a]

→ neural memory 의 실이득이 어느 규모 · 태스크에서
확실한지는 아직 합의 없음 [a]

Part II

Scaling 병목 3 종

model-size scaling — 파라미터가 아니라 '용량'

- TTT/Titans 는 문맥을 '파라미터'가 아니라 '메모리 용량'으로 저장한다
 - 용량 이론 : d^2 파라미터를 가진 메모리에 $\sim d$ 규모의 정보만 안정 저장 (associative/Hopfield) [b]
- Chunk mismatch: 청크 단위 갱신이 규모에서 성능을 열화 (Titans Revisited) [a]
 - 고정 용량 recurrent state 의 긴 문맥 정보이론적 벽 — 길수록 간섭 누적 [b]
- hybrid attention 비율 의존 + scaling-law 상 표현력↔효율 tradeoff [a]
- → 단순히 파라미터를 키운다고 문맥 기억이 비례해 늘지 않는다 [b]

4 · 병목 ②

training parallel/batch scaling

- inner-loop 갱신의 순차 의존 → 시퀀스 방향 병렬화 (sequence-parallel) 가 어렵다 [b]
 - 작은 online minibatch(16-64 토큰) → GPU FLOPs 이용률이 흔히 < 5% [a]
- backprop-through-the-inner-loop: 갱신 궤적을 미분하는 학습의 메모리 · 계산 비용 [b]
 - batch × state footprint: 표현력 위해 state 를 키우면 학습 배치가 늘린다 [b]
- 비선형 recurrence 의 병렬화는 가능하나 조건부 (수렴 · 안정성) [b]
- → 알고리즘이 아니라 '하드웨어 활용률' 이 학습 스케일의 병목 [b]

serving batch inference + Prefill/Decode(P/D)

- batching 은 ' 요청들이 공유하는 static weight' 를 전제 — TTT 는 이를 깬다 [a]
 - request 마다 mutable fast-weight state(low-rank delta/learner state) 를 소유 [a]
- recurrent state 는 KV cache 처럼 prefix-cache(재사용) 되지 않는다 [a]
 - prefill(대량 병렬 흡수) vs decode(순차 갱신 , state 대역폭 bound) 의 비대칭 [b]
- hybrid(attention+recurrent) 에서 P/D disaggregation 이 더 어렵다 [b]
- → 모델은 빨라도 ' 동시 요청 배치 ' 가 안 되면 서빙 처리량이 안 는다 [b]

Part III

해법 3 종 — 무엇을 · 어떻게 · 왜 working

model-size — 규모를 '물려받고' 용량을 키운다

무엇을 · 어떻게

규모 상속 : 큰 Transformer 를 distill/linearize

MOHAWK · Llama · LoLCATs(405B) · Liger [a]

용량 확장 : Atlas 다항 feature, sparse/expandable state

성장 : Sleep(param expansion+Knowledge Seeding

+Dreaming), HOPE nested depth

hybrid: attention 일부 남겨 규모 병목 우회

왜 working

규모는 사전학습된 Transformer 의 것을 물려받아

scratch 대규모 학습 없이 405B 급 도달 [a]

용량은 feature/state 차원을 키워 $d^2 \rightarrow d$ 벽을 완화 [b]

성장은 필요할 때 파라미터를 늘려 고정 용량을

넘어섬 (offline consolidation) [a]

한계 : distill 은 '남의 규모' 에 의존 [b]

training — 순차성을 chunk· 병렬화로 접는다

무엇을 · 어떻게

LaCT: 극단적 large chunk(2K-1M 토큰) [a]
→ util 을 orders-of-mag 개선,
nonlinear state 를 파라미터의 40% 까지 [a]
chunkwise-parallel: DeltaNet WY, TNT 계층
비선형 recurrence 병렬화: DEER→ParaRNN,
Predictability (Newton/fixed-point/scan)
커널: Comba(Triton, +40%), Tiled FLA

왜 working

큰 chunk 는 갱신을 compute-bound 로 만들어
TensorCore 를 채운다 (util ↑) [a]
WY/householder 는 delta 재귀를 하나의
행렬 연산으로 재작성 → chunk 병렬 [a]
비선형 병렬화는 순차 재귀를 병렬 solve 로 대체 [b]
trade-off: 큰 chunk 는 청크 내 세밀 적응을 지연 [b]

serving — 배치 · 캐시 프리미티브를 재설계

무엇을 · 어떻게

RW-TTT: owner/version/READ-WRITE 태깅으로

호환 phase 만 batch, write 는 owner 에만 commit [a]

hybrid prefix caching: Marconi · HYPIC · Sparse Prefix

IO-aware 서빙 : KVBuffer

최소 state: In-Place TTT(별도 state 제거)

hybrid stack: Nemotron · Kimi Linear (KV ↓)

왜 working

RW-TTT: In-Place-TTT 8 스트림에서 274.61 tok/s

= 순차 $9.31\times$ · 동일메모리 복제 $3.44\times$ [a]

재사용 프리미티브를 recurrent state 에 맞게 재정의

→ Marconi 최대 $34.4\times$ 캐시 이득 [a]

In-Place 는 요청별 state 부담 자체를 줄임 [b]

한계 : hybrid prefix-cache 표준은 아직 없음 [c]

Part IV

memory device 집중 조명

한계 — 하나의 장치, 두 상한

(i) 신경 메모리 (저장 장치로서)

유한 용량 : d^2 파라미터에 $\sim d$ 정보 (Hopfield/assoc) [b]

interference: key 충돌 시 덮어쓰기 · 간섭 [b]

retrieval error 가 저장량 증가에 따라 성장 [b]

고정 용량 → 긴 문맥에서 정보이론적 열화 [b]

(ii) physical memory (HBM)

state footprint 의 HBM 점유 [b]

decode 가 state 읽기의 memory-bandwidth 에 bound [b]

batch $\times$ state → 동시성의 메모리 상한 [b]

→ 세 병목 (model-size · training · serving) 이
만나는 물리적 결절점 [b]

개선 — 더 크게 · 더 정확히 · 더 싸게

- 더 크게 (용량 ↑ , FLOPs 는 묶음): Sparse State Expansion · Sparse Delta Memory [a]
- 더 정확히 (압축 대신 정확 저장): exact memory — Hippocampus for Linear Attention · EFLA [a]
- 자라는 메모리 : KV-Means · Memory Caching(growing) [a]
 - 간섭 ↓ : erase-write 주소 분리 (Gated DeltaNet-2, Erase-then-Delta) [a]
 - 용량 자체 ↑ : 고용량 feature map(Atlas 다항 · sympow); 압축 (Lattice · Trellis · LoLA) [a]
- 물리 개선 : TTQ(양자화로 footprint ↓) · KVBuffer(대역폭) · In-Place(별도 state 제거) [a]
- → 각 개선 축이 특정 병목을 직접 겨냥한다 [b]

병목 × 해법 × memory device 지도

병목	근본 원인	대표 해법	memory device
① model-size	용량 $d^2 \rightarrow d$	distill · 용량확장	용량 상한
② training	순차 $\rightarrow$ util<5%	큰 chunk · 병렬화	state footprint
③ serving	request-owned state	RW-TTT · prefix cache	대역폭 · footprint

관통하는 한 줄 : 모든 효율 이득은 결국 memory device 의 유한한 용량 · 대역폭과 거래한다 .

프론티어 = 큰 chunk(LaCT) · 최소 state(In-Place) · 요청별 서빙 (RW-TTT) 을 ' 동시에 ' — 아직 갈래는 분리돼 있다 .

무엇이 아직 안 풀렸나

- 큰 chunk(util) vs 세밀한 순차 적응 (정확도) 을 동시에 ? chunk 내부 의존 손실의 이론 상한 [c]
- $TTT \approx \text{linear attention}$ 등가라면 , 'test-time 학습 ' 의 표현력 우위는 실제로 어디서 오나 ? [c]
- $d^2 \rightarrow d$ 용량 벽 : sparse/expandable 이 상수 개선인가 , 스케일 지수 자체를 바꾸나 ? [c]
- request-owned state 서빙이 수천 동시 요청에서 스케일하나 ? (메모리 폭발) [b]
- hybrid recurrent state 의 표준 prefix-cache 프리미티브는 ? [c]
- model-size 가 distill 로만 풀린다면 , scratch 대규모 학습 경로는 닫힌 것인가 ? [c]
- 재현성 (Titans Revisited) — 실이득이 어느 규모 · 태스크에서 확실한가 ? [a]
- 물리 (HBM) × 알고리즘 (state) co-design(HATIR 류 비용모델) 로 무엇을 예측하나 ? [c]

한 장 요약

- TTT/Titans 의 표현력 = ' 순차적 test-time 갱신 ' — 바로 그 순차성이 모든 병목의 뿌리
 - 세 병목은 한 뿌리의 세 그림자 : 파라미터로 안 커지고 · 학습이 병렬화 안 되고 · 서빙이 배치 안 됨
 - 해법 세 방향 : 규모 상속 (distill) · 순차성 접기 (큰 chunk · 병렬화) · 프리미티브 재설계 (서빙)
- memory device = 세 병목이 만나는 물리적 결절 ; 개선 3 축 = 더 크게 · 더 정확히 · 더 싸게
 - 근거 : study paper(research/ttt-titans-efficiency-study.md) + efficient-TTT 93 편 코퍼스